

Convolutional Neural Networks (CNN) - Object Recognition

✓ Imports

```
from numpy.random import seed
seed(888)

#from tensorflow import set_random_seed
#set_random_seed(4112)
import tensorflow
tensorflow.random.set_seed(112)
```

```
import os
import numpy as np
import itertools

import tensorflow as tf
import keras
from keras.datasets import cifar10 # importing the dataset

from keras.models import Sequential #to define model/ layers
from keras.layers import Dense, Conv2D, MaxPool2D, Flatten

from sklearn.metrics import confusion_matrix

# To Explore the images
from IPython.display import display
from keras.preprocessing.image import array_to_img

from tensorflow.keras.utils import to_categorical

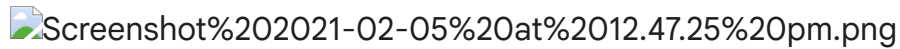
import matplotlib.pyplot as plt
%matplotlib inline
```

```
import pandas as pd
```

We are using Tensorflow to power Keras

✓ Get the Dataset

CIFAR-10 is an established computer-vision dataset used for object recognition. It is a subset of the 80 million tiny images dataset and consists of 60,000 32x32 color images containing one of 10 object classes, with 6000 images per class. It was collected by Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton. The dataset is popularly used to train image classification models



```
# Getting the dataset as a Tuple
```

```
(x_train_all, y_train_all), (x_test, y_test) = cifar10.load_data()
```

```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python
152125440/170498071 5:29 18us/step
```

✓ Constants

```
LABEL_NAMES = ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog',
```

✓ Exploring the Data

Lets look at the first image in the dataset

```
x_train_all.shape
```

```
x_train_all[0]
```

```
x_train_all[0].shape
```

✓ Using ipython to display the image

```
# To use the ipython display to view an image
```

```
pic = array_to_img(x_train_all[0])
display(pic)
```

✓ Using Matplotlib to view the image

```
plt.imshow(x_train_all[0])
```

```
# To check the label
y_train_all.shape
```

```
# Note that in the image above the index 1 corresponds to "Automobile"
# we have a 2 dimension numpy array; that is why we also include " [0
y_train_all[0][0]
```

```
# Using the lable names to get the actual names of classes
LABEL_NAMES[y_train_all[0][0]]
```

✓ The shape of the image

```
* 32, 32 is the weight and the height
* 3 is the number of channels (These are the number of colors): Red, Green, Blue
```

- `x_train_all.shape >>> (50000, 32, 32, 3)`
 - this means we have 50,000 entries | then 32x32 weight and height| 3 colors (RGB)

```
x_train_all.shape
```

```
number_of_images, x, y, c = x_train_all.shape
print(f'Number of images = {number_of_images} \t| width = {x} \t| height = {y} \t| channels = {c}')
```

```
x_test.shape
```

✓ Preprocess Data

* We need to preprocess our data so that it is easier to feed it to our neural network.

✓ Scalling both x_train and test

```
x_train_all = x_train_all / 255.0
```

```
x_test = x_test / 255.0
```

```
y_test
```

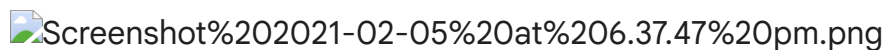
✓ Creating categorical encoding for the "y" data

```
# 10 >>> simply means we have 10 classes like we already know (creating y_cat_train_all = to_categorical(y_train_all,10)
```

```
# 10 >>> simply means we have 10 classes like we already know (creating y_cat_test = to_categorical(y_test,10)
```

```
y_cat_train_all
```

✓ Creating the Validation dataset



For small data we usually go with: * 60% for Training * 20% Validation * 20% Testing

Only the final selected model gets to see the testing data. This helps us to ensure that we have close to real data in real-world when the model is deployed. Only our best model gets to see our testing dataset. Because it will give us a realistic impression of how our model will do in the real world

However, if the dataset is enormous.: * 1% for is used for validation * 1% for is used for testing

```
VALIDATION_SIZE = 10000
```

```
# VALIDATION_SIZE = 10,000 as defined above  
  
x_val = x_train_all[:VALIDATION_SIZE]  
y_val_cat = y_cat_train_all[:VALIDATION_SIZE]  
x_val.shape
```

```
y_val_cat
```

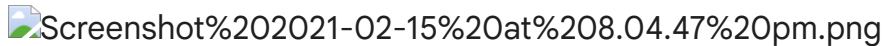
NEXT:

- We Create two NumPy arrays `x_train` and `y_train` that have the shape(40000, 3072) and (40000,1) respectively.
- They will contain the last 40000 values from `x_train_all` and `y_train_all` respectively

```
x_train = x_train_all[VALIDATION_SIZE:]
y_cat_train= y_cat_train_all[VALIDATION_SIZE:]
```

```
x_train.shape
```

```
y_cat_train
```



NOTE:

* *FILTERS*: Typical values for the number of filters can be determined by the data set's complexity. So essentially the larger the images, the more variety and the more classes you're trying to classify then the more filters you should have.

* Most times people typically pick filter based on powers of 2, for example, 32. However, if you have more complex data like road signs etc. you should be starting with a higher filter value

The default STRIDE value is 1 x 1 pixel

✓ BUILDING THE MODEL

```
model = Sequential()

## ***** FIRST SET OF LAYERS *****

# CONVOLUTIONAL LAYER
model.add(Conv2D(filters=32, kernel_size=(4,4),input_shape=(32, 32, 3
# POOLING LAYER
model.add(MaxPool2D(pool_size=(2, 2)))

## ***** SECOND SET OF LAYERS *****
#Since the shape of the data is 32 x 32 x 3 =3072 ...
#We need to deal with this more complex structure by adding yet another

# *****CONVOLUTIONAL LAYER
model.add(Conv2D(filters=32, kernel_size=(4,4),input_shape=(32, 32, 3
# POOLING LAYER
model.add(MaxPool2D(pool_size=(2, 2)))

# FLATTEN IMAGES FROM 32 x 32 x 3 =3072 BEFORE FINAL LAYER
model.add(Flatten())
```

```
# 256 NEURONS IN DENSE HIDDEN LAYER (YOU CAN CHANGE THIS NUMBER OF NEURONS)
model.add(Dense(256, activation='relu'))

# LAST LAYER IS THE CLASSIFIER, THUS 10 POSSIBLE CLASSES
model.add(Dense(10, activation='softmax'))

model.compile(loss='categorical_crossentropy',
              optimizer='adam',
              metrics=['accuracy'])
```

```
model.summary()
```

✓ Adding Early stopping

```
from tensorflow.keras.callbacks import EarlyStopping
```

```
early_stop = EarlyStopping(monitor='val_loss',patience=2)
```

```
history = model.fit(x_train,y_cat_train,epochs=25,validation_data=(x_test,y_cat_test),callbacks=[early_stop])
```

```
model.history.history.keys()
```

```
metrics = pd.DataFrame(model.history.history)
```

```
metrics
```

```
metrics[['loss', 'val_loss']].plot()
plt.title('Training Loss Vs Validation Loss', fontsize=16)
plt.show()
```

```
metrics[['accuracy', 'val_accuracy']].plot()
plt.title('Training Accuracy Vs Validation Accuracy', fontsize=16)
plt.show()
```

✓ Validating on Test Data

```
model.evaluate(x_test,y_cat_test)
```

✓ Classification Report and Confusion Matrix

```
from sklearn.metrics import classification_report, confusion_matrix
```

```
#predictions = model.predict_classes(x_test)
predictions = np.argmax(model.predict(x_test), axis=-1)
```

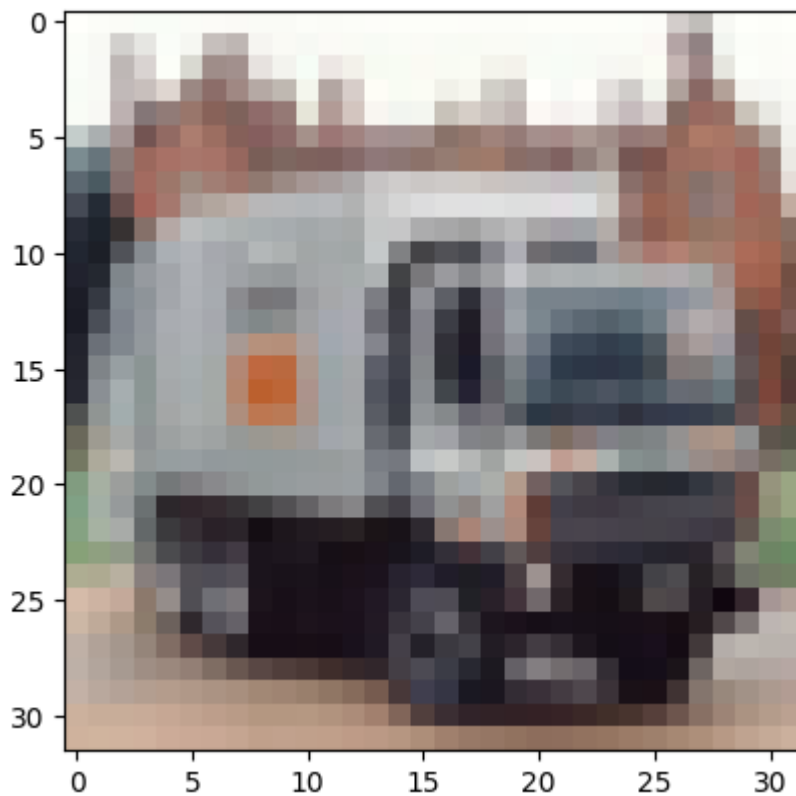
```
print(classification_report(y_test,predictions))
```

```
confusion_matrix(y_test,predictions)
```

✓ Predicting on single image

```
plt.imshow(x_test[11])
```

```
<matplotlib.image.AxesImage at 0x7f2db7713fe0>
```



```
my_image = x_test[11]
```

```
# SHAPE --> (num_images,width,height,color_channels)
model.predict_classes(my_image.reshape(1,32,32,3))
```

```
import numpy as np
```

```
prediction = np.argmax(model.predict(my_image.reshape(1,32,32,3))), ax
print("Predicted class:", prediction[0])
```

```
1/1 ————— 0s 80ms/step
Predicted class: 9
```

```
LABEL_NAMES[y_test[16][0]]
```

✓ Conclusion

The CNN model was successfully run, and the different stages of the algorithm, including data preprocessing, model training, evaluation, and prediction, were reviewed. The input image was changed from `x_test[16]` to `x_test[12]` for the testing phase. The model predicted class 9 (a truck), which matched the displayed image, demonstrating that the CNN correctly recognised the object. This shows that the trained model is able to accurately classify unseen images from the CIFAR-10 test dataset.

Start coding or generate with AI.